

SKILLSBENCH Tasker

SkillsBench - Tasker Playbook

Authors Apurv Dwivedi Muneeb Hashmi Omar Adel

Note

This document serves as a documentation of the SkillsBench Data Generation Guidelines for the *Taskers*. This documentation was created at Jun 26, 2026 . Last update was Jul 1, 2026 .

Any versioning on any of the sections of this document refers to the level of experimentation we are on. For instance, v3 means that this is the third experiment with this specific point. On average, each experiment takes around two weeks to complete.

Another important piece of information is this deck  [INTERNAL] SkillsBench Introduction

Prerequisites

This document assumes that you already have knowledge of **Supervised Fine-Tuning** and the **SkillsBench** research paper.

Introduction

What is a Skill?

A skill is a markdown file that tells the model how to behave for a specific capability domain.

↗ **Go deeper:** [skill.md](#) for more on Skills file anatomy, structure and guidelines.

How can a model use a Skill?

Skills live in a registry.

The registry is the model's skill library: indexed, discoverable, and always available.

- A structured store of all available skill .md files
- Each skill has metadata: name, domain, description; enough for the model to identify relevance
- The model reads registry metadata to decide which skill applies
- Skill use can sometimes be manual (forced)

Why use Skills?

Skills make model behavior targeted, auditable, and improvable by domain. We don't have to rely on the model training data to cover every existing use-case.

What is Skills Benchmarking?

Skills benchmarking evaluates the model's skill usage, not just its output quality.

Two distinct questions being tested:

- Judgment: did the model correctly identify that a skill was needed, and pick the right one?
- Compliance: did the model faithfully follow the skill's .md instructions once it pulled it?

A model can fail on either axis independently — both matter

What is a good task for Skills Benchmarking?

Good benchmark tasks are built to test specific skill instructions compliance.

- Include tasks where the skill's instructions are easy to shortcut or overlook
- Avoid tasks where the model can produce a correct-looking output without actually using the skill
- Each task should have a clear, auditable expected behavior tied to the skill

Why does Skills Benchmarking matter?

AI providers need to ensure that their model adapts to this new paradigm of using existing skills and train their models on this data to fill in any possible gaps in:

1. Skill instruction following
2. The judgement on what triggers a skill-use decision

Tools & Links

Important tools and links you would need in your task generation.

Tool	Use
SkillsBench Labeling Tool	Our tool for generating SkillsBench samples
Grammarly	Used for linguistic checks.
JSON Formatter	Formatting JSON structures.
XML Formatter	Formatting XML structures.
CSV Table Formatter	Formatting CSV Tables.
Name generator	Generates unique realistic names for different entities.
Skills.md	Reference for skills anatomy
https://www.skillsbench.ai/	Reference for the paper (continuously updated)

Onboarding Resources

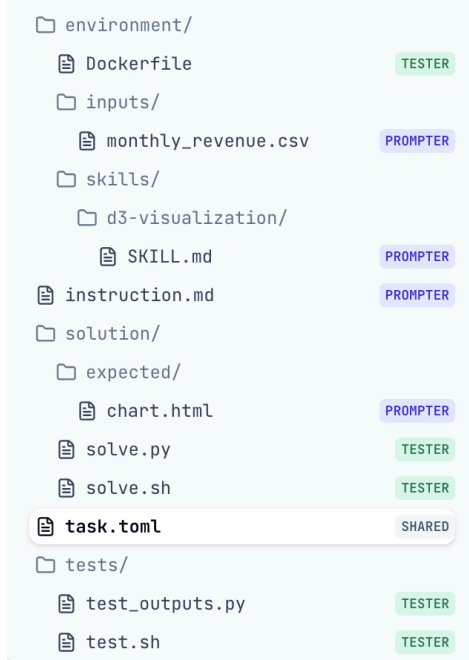
#	Resource	Tier	Time	Link
1	Tasker Workflow	Mandatory	0.5h	slides
2	SkillsBench paper	Mandatory	1.5h	PDF
3	Onboarding video	Mandatory	3.0h	recording
4	Annotator training doc	Optional	-	doc
5	Calibration set / client samples	Optional	-	dashboard
6	UI Navigation Guide	Mandatory	0.5h	doc

Questions or blockers during onboarding, go to your Pod Lead.

Task Anatomy

What is a SkillsBench task?

A task is a fully self-contained evaluation package in [harbor format](#).



environment/	
Dockerfile	TESTER
inputs/	
monthly_revenue.csv	PROMPTER
skills/	
d3-visualization/	
SKILL.md	PROMPTER
instruction.md	PROMPTER
solution/	
expected/	
chart.html	PROMPTER
solve.py	TESTER
solve.sh	TESTER
task.toml	SHARED
tests/	
test_outputs.py	TESTER
test.sh	TESTER

Note: The existing tooling removes the hassle of managing this format from you. You only need to worry about creating a good prompt, evaluating it properly and creating a correct golden solution.

Roles

Tasker (this is you!)

Creates the prompt, golden solution and verifiers.

Tester

Converts tasker-generated verifiers into deterministic checks in Python.

Reviewer

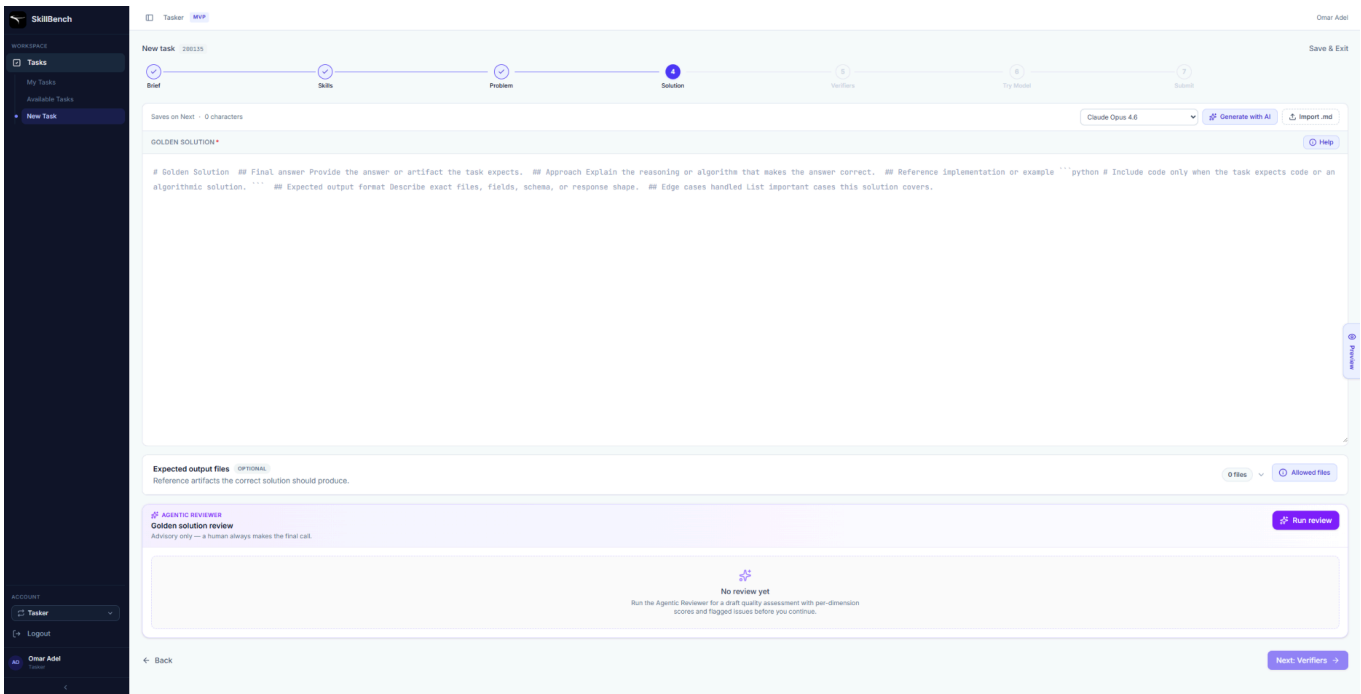
Ensures the quality of the prompt and that model failure report aligns with the project requirements.

Evaluator

Runs the model evaluation report and either ships or rejects the task.

Building the task

Task Creation Tool



Brief → Skills → Problem → Solution → Verifiers → Try Model → Submit

#	Stage	What you do
1	Brief	Name the task, describe it, pick the domain and difficulty .
2	Skills	Select the skill(s) the task will be tested <i>with</i> vs. <i>without</i> .
3	Problem	Write the prompt the agent reads + upload the input files it gets at runtime.
4	Solution	Write the golden (reference) solution + attach the expected output artifact(s).
5	Verifiers	Add the deterministic PASS/FAIL checks that decide correctness.
6	Try Model	Run the models with & without skills to confirm the skill gives real uplift.
7	Submit	Final review, then submit. The Tester team runs the authoritative evaluation afterward.

Move forward with **Next** (bottom-right); jump between completed stages from the top progress bar; **Save & Exit** any time (progress auto-saves).

↗ **Go deeper:** [UI Navigation Guide](#) — full screenshot-by-screenshot walkthrough of every stage.

Task Creation Thought Process

1. Decide your task:
 - a. What's the domain and the use case?
 - b. Read through the skill and figure out what you can challenge the model on
 - c. Decide whether you want the agent to build something from scratch or operate on input artifacts that you're going to upload (and prepare those artifacts)
 - i. Make sure the artifacts are realistic and factually correct w.r.t the domain you've selected
 - d. Scope of complexity
 - e. Capability testing with skill and domain
2. Write the prompt and upload the input artifacts
3. Write the verifiers for initial testing step
4. Golden solution: should be grounded requirement from agent - aligned to verifiers
5. Test the model using the verifiers and test the model failure modes and qty
6. If the task meets the acceptance criteria, submit for review

Project Requirements

For the scope of this project, we run a comparison test on two models:

1. Claude Opus 4.6 (in Claude-Code Harness)
2. Qwen 3.7 Max (in Open-Code Harness)

Each task is run by the same model **twice**:

- **with-skills** — the agent gets the skill-registry before it starts.
- **without-skills** — identical, but the skill-registry is removed.

The score *difference* between the two is the entire measurement. So:

The one rule: a good task *fails without skills*. If a strong model can solve it from the prompt alone, the task is worthless. We need to build around a genuine domain convention or edge case where a capable generalist model gets wrong.

Finding a model-breaking task: start from the skill, then build around it. You're choosing skills from a fixed catalogue (Stage 2),

1. Read the skill closely,
2. Figure out the exact skill, convention or edge case it teaches
3. Write a prompt (Stage 3) that can't be done *correctly* without knowing that skill.

A model without the skill should get it subtly wrong; a model with it should get it right (mostly). The catch is that your prompt says *what* you want, never *how* to do it — so the task has to depend on the convention without you ever spelling the convention out (more on that in Stage 3).

↗ **Go deeper:** [SkillsBench research paper](#) (skim it at least to get a background — could use an LLM to help).

Building the task, stage by stage

These seven stages are the wizard itself. Do them in order; each one below covers what goes in and the mistakes that tend to get it bounced back.

Stage 1 — Brief

The task's identity card.

- **Task name** — unique and descriptive (e.g. ecommerce-sales-analysis-01). Must be unique across the benchmark.
- **Description** — one or two outcome-focused sentences on what the agent must accomplish.
- **Domain** — the primary domain (e.g. Software Development). **This is the gatekeeper for Stage 2:** it controls which skills appear. If a skill you need is missing later, come back and fix the domain.
- **Difficulty** — **Core**, **Extended**, or **Extreme**, by how challenging the task is meant to be. No strict formula — most first tasks sit at **Core/Extended**; if unsure, set your best guess and confirm the level with your Pod Lead.

Stage 2 — Skills

Pick the skill file(s) that will be used in the **with-skills run**. Only skills mapped to your chosen domain appear (the list is labelled for that domain). Use the tabs (**Individual**, **My Skill Copies**, **Skill Combo**) or search; tick to select (chosen skills show as chips). **View & Edit** opens the skill's metadata (ID, version, mapped domains, tags) and its raw SKILL.md — confirm the procedure genuinely matches your task before committing.

For now you only select from the predefined skill catalogue — you don't write your own skills. Use **View & Edit** to *inspect* a skill and confirm it fits your task. (Authoring and editing your own skills is on the roadmap; at this stage your job is to choose the best existing one.)

How many to pick: prefer **1–3 focused skills** that each pull real weight. More is not better — the research shows 2–3 well-scoped skills give the strongest uplift, while piling on 4+, or picking a long "comprehensive" documentation-style skill, actually *reduces* (and can reverse) the gain. A short, sharply-relevant skill beats an encyclopedia.

What makes a skill good (the bar for judging which catalogue skill to pick):

- **Procedural, not factual** — workflows, decision trees, ordered steps, worked-example *patterns*. Not a glossary or tool docs.
- **Class-of-tasks, not single-instance** — it should help on a *sibling* task too, not only yours. If it only makes sense with your one task, it's a hint, not a skill.
- **Generic** — code snippets are patterns, never the literal commands/values that solve your task.
- **About the traps** — the conventions, ordering, unit assumptions, and edge cases a strong model routinely gets subtly wrong.

 **LEAKAGE = REJECTION.** A skill must never contain: task-specific filenames/paths/identifiers · exact command sequences that solve the task · constants/magic numbers from the answer · references to specific expected outputs or your task's unique field/sheet names.

	Example	Why
 Good	"When building a source-data sheet, use one header row with no merged cells, one record per row, and no subtotal rows mixed into the data. Join	Reusable convention; works on any task in the class; leaks nothing.

	<i>all sources before building summaries."</i>	
✗ Bad	<i>"The SA2 codes in households.pdf join to column B in spending.xlsx. The expected number of rows is 2,042."</i>	Names task files, a task-specific column, and a magic number from the answer — instant reject.

Stage 3 — Problem statement (prompt/instruction.md)

Write it like a **real stakeholder brief**, not a tutorial. Natural tone is good; robotic/templated prompts get rejected. You can type it in the editor or **Import .md**. Then upload any **runtime context files** the agent reads (data, fixtures — e.g. config.json, trace.json).

This is where you exploit the skill (§1). Make at least one requirement hinge on the skill's convention or edge case — stated as a plain *what* (an output shape, value rule, or constraint), so a model that lacks the convention produces a subtly wrong answer. The need for the skill comes from the requirement; the method itself never appears in the prompt.

Hit all five marks:

1. **Explicit output path** — always `/root/<filename>`.
2. **Structured requirements** — name the input files; spell out the output shape (sheets, JSON keys, fields, format).
3. **Constraints listed** — date formats, rounding, units, allowed values, naming, anything forbidden.
4. **Context-first** — one sentence of business context, then the requirements.
5. **Every requirement testable** by ≥1 verifier check in Stage 5 (else rewrite or drop it).

Never include: how-to hints · library/tool names · expected values/results · anything that belongs in the skill · conditions on fields that don't exist · **any skill name or "use the right skill" wording** (the agent must discover the skill on its own).

	Example	Why
✓ Good	<i>"Save the completed workbook as /root/retail_spend_profile.xlsx. The first four sheets must be real Excel pivot-table objects, not pasted summaries. Use the stable regional identifier shared by the two input datasets to combine the records."</i>	Says <i>what</i> + exact path + a constraint; names no tools; hints no method.
✗ Bad	<i>"Use pandas to read the PDF. Merge the dataframes on the SA2 code column. Use openpyxl to create pivot tables."</i>	Leaks the method , names tools , and exposes a task-specific column.

Mini-checklist:

- ✓ output-only
- ✓ exact `/root/` path
- ✓ all constraints listed
- ✓ no method/tool/skill hints
- ✓ every requirement testable
- ✓ input files uploaded

Stage 4 — Golden solution

In the platform you capture two things here: the **golden solution** (your written reference approach) and the **expected output file(s)** it produces. What the platform *can't* store is the *how* — the reproducible recipe that turns the inputs into that output. That goes in the tracker (see **Solver files** below).

- **Golden solution** — write (or **Import .md**) the reference approach and key implementation details: the steps, the conventions you follow, the edge cases you handle. You may use **Generate with AI** for a *draft*, but the canonical solution has to be human-verified.
- **Expected output file(s)** — attach the artifact the correct solution produces (e.g. `report.json`). Every task we build has one.

The solution must be *genuinely correct*, not "verifier-passing by construction." Write it the way a domain expert actually would. If you'd have to bend the solution to satisfy a quirk of a verifier, the **verifier is wrong** — fix the verifier in Stage 5.

Solver files — how you reached the golden solution. The golden solution is the *result*; the solver file is the *recipe a human would follow to reproduce it*. The platform **currently** has nowhere to store this, so it lives in the tracker instead:

- **Technical tasks** (software, most data analysis — input → output can be produced by code): add a `solve.py` / `reference_solution.py` (or a `.sql/db` file, whatever fits) that reads the inputs and writes the output deterministically. This is *your* reference implementation, not something a model generates.
- **Non-technical tasks** (some maths/science, where no script maps input → output): add a `steps.md` that spells out the exact steps to reproduce the golden solution by hand. (The Tester team turns `steps.md` into a solver later.)
- Rule of thumb: include **whatever a human would need to reproduce the golden solution** — code, steps, or both.

Where they go: make a folder in this [shared drive](#) - **name it with the task ID**, put the solver file(s) inside, and paste the folder link into the tracker's **Solver Files** column (**column K**).

↗ **Example:** a filled-in [solver folder](#).

You don't run the solution to a score here — after you submit, the **Tester team** runs the authoritative oracle evaluation. Your job is to make the golden solution, its expected output, and the solver recipe all *correct*.

Stage 5 — Verifiers

The verifier is the most important thing you build here: it's where your spec turns into checks a machine can run. Each assertion has a **Category** (the rubric bucket it proves, e.g. Output Artifact, Structure and Schema, Value Correctness), a **PASS/FAIL** condition, and a **natural-language assertion** (one concrete, checkable statement per row). The header tracks totals (e.g. *21 assertions · 21 valid · 0 invalid*).

The part people get wrong: it's not enough that your golden solution passes. The verifier also has to **reject a wrong-but-plausible answer**. For every check, ask yourself "*what incorrect output would also slip past this?*" If you can name one, you've found a hole.

The six standards (every check satisfies all six):

1. **Objective** — a deterministic yes/no. No ratings, no "is it reasonable?", no LLM-as-judge.

2. **Grounded** — traces to something in the prompt or the skill. No new requirements introduced here.
3. **Deterministic** — no randomness, clock, or live network. Same input → same result.
4. **Full coverage** — every prompt requirement and every skill promise → ≥ 1 check (incl. the "boring" ones: file exists, right format/extension, right /root/ location).
5. **No phantom conditions** — never reference a field/file/key/sheet that doesn't exist.
6. **Negative tests** — every "do not do X" rule gets a check that **fails** if X appears.

Mistakes reviewers catch most:

- **Artifact drift (the #1 rejection)** — the prompt, skill, expected output, and verifier disagree on a filename, schema, threshold, or number (a typo'd path, a phantom extra column). Before submitting, do a **consistency pass**: reconcile all four character-for-character.
- **Overfitting** — hard-coding a value true only for *your* golden file (a row count, an ID). **Recompute** it from the inputs instead.
- **Process not outcome** — checking *how* it was solved ("does it normalize text first?"). Test the observable consequence in the output.
- **Weak validation** — only checking shape, never comparing *values* to ground truth. A confidently-wrong output passes.
- **Phantom checks** — asserting fields the prompt never mentioned.
- **Missing tolerances** — exact float equality (fails correct answers) or none at all (passes wrong ones). State an explicit tolerance on every numeric check.

Anchor every assertion to an exact file, key, value, or count (e.g. "*Does per_request have exactly 20 entries?*") — never vague ("*Is the answer reasonable?*").

Worked example — same task, weak → strong:

	Assertion	Problem / Fix
✗	Does /root/matched_payess.csv exist?	Filename typo won't match the prompt.
✗	Does the implementation normalize names before matching?	Process check — invisible in the output.
✗	Does the output contain exactly 4 blank rows?	Overfit to the golden file.
✗	Are the vendor names plausible for each payee?	Subjective / LLM-judge.
✓	Does /root/matched_payees.csv exist with exactly columns id, payee, matched_vendor, match_confidence (in order)?	Exact name + schema; no phantom.
✓	Against the ground-truth crosswalk, do matched_vendor values agree on a 25-row sample covering abbreviation/suffix/ambiguous/unmatched cases?	Deterministic value correctness vs ground truth.
✓	Is every blank match a row whose best score is below the prompt's stated threshold, recomputed from inputs?	Tests the rule , not a memorized count.

	Does no row carry a vendor absent from the master list?	Negative test.
---	---	----------------

Verifier self-review checklist

Run this on your checks before you move on:

Grounding & consistency

- ☐ Every filename, path, column, and number in a check appears **verbatim** in the prompt or skill.
- ☐ Prompt ↔ skill ↔ expected output ↔ verifier agree on paths, schema (names/count/order), and every threshold.
- ☐ No check references a field, file, key, or sheet that doesn't exist.

Correctness & strength

- ☐ At least one check compares output **values** to ground truth (recomputed from inputs, or loaded from a file the agent can't read).
- ☐ A wrong-but-plausible output would **fail** ≥1 check (try to picture one).
- ☐ No check tests *how* it was solved — every check is judgeable from the output alone.
- ☐ No expected value is hard-coded from the golden file when it could be recomputed.

Coverage & shape

- ☐ Every prompt requirement and skill promise maps to ≥1 check — incl. the boring ones (file exists, right format/extension, /root/ location).
- ☐ Every "must not / only / exactly" rule has a negative test that fails when violated.
- ☐ No two checks assert the same thing; keep the suite focused (~under 10 unless justified).

Determinism & numerics

- ☐ No LLM-as-judge, randomness, clock, or live network anywhere.
- ☐ Every numeric check has an explicit tolerance, and expected values came from running code — not from your head.

↗ **Go deeper:** the verifier section (Part 6) of the [Annotator training doc](#).

Stage 6 — Try Model (prove it's model-breaking)

Why: to confirm the task is hard on its own *and* that the skill is what closes the gap — before you submit.

What "model-breaking" means: a model is *broken* by the task when it **fails at least one verifier** — it does **not** pass all of them. The whole point is a task that no model can fully solve on its own.

The part below is deprecated; the only requirement as of now is that the task should be complex enough - for details, please refer to [this thread](#).

~~**Model-pass criteria** (read these off the per-run verifier counts in Try Model — all four must hold):~~

- ~~• **Qwen 3.7 Max never passes all verifiers** — with or without skills:~~
- ~~• **Qwen's passed-verifier count stays strictly below Claude's** — in the same condition, never equal, never higher.~~
- ~~• **Claude Opus 4.6 may pass all verifiers with skills; Qwen must not.**~~

- ~~Skill uplift on both models~~ — adding the skill raises each model's passed-verifier count by **at least 4**.

Example — a 20-verifier task that clears the bar:

Model	Without skills	With skills
Claude Opus 4.6	6 / 20	20 / 20
Qwen 3.7 Max	4 / 20	12 / 20

~~Qwen never passes all (4, 12 < 20) ✓ Qwen < Claude in both conditions (4 < 6, 12 < 20) ✓ Claude passes all with skills; Qwen doesn't ✓ uplift ≥ 1 for both (Claude +14, Qwen +8) ✓ → passes.~~

How to run it: pick a model, toggle **Without / With skills** (or **Compare both**), hit **Generate Response**; do both **Opus 4.6** and **Qwen 3.7**, without skills first, then with. Try Model is a **directional check** — enough to see the signal; the **Tester team** re-runs it across **5 attempts per config** afterward (their authoritative gate also caps Qwen-with-skills at **≤ 2/5 attempts, ≤ 50%**), so don't sweat reproducing exact counts here. The summary cards (**Without Skills, With Skills, Pass Uplift, Rate Change, Improved, Regressed**) give a quick read on whether the skill is helping; expand any verifier row (**Without skills / With skills** toggle) for **Observed / Evidence / Recommended action / Judge response**, and open the **Response** tab for the full trajectory.

~~If without skills already passes, the task isn't skill-dependent enough. If with skills passes too easily, it's too easy, or the skill is leaking. What you want: **fails without the skill, and succeeds with it — but not effortlessly.**~~

Stage 7 — Submit

Re-check each stage from the top progress bar — all should show completed — then submit. After submission, the **Tester team** runs the authoritative oracle/evaluation; you do **not** need to confirm a score yourself.

Also make sure that you have submitted the solver files folder in this [drive folder](#) (more about [here](#)).

Agentic Reviewer (AR) — self-review before you submit

AR is an automated reviewer built into the platform. Run it on your work section by section before you submit — it catches the same kinds of issues a Pod Lead would otherwise send back, so clearing AR first saves you a review round-trip.

What AR checks for Taskers: your **Problem Statement, Golden Solution, and Verifiers**, each reviewed on its own, plus a **Domain Review** that looks at all three together for consistency.

How to use it (AR is *not* a guess-and-check loop):

- **Run it on a *finished* section, not mid-edit.** Complete the Problem Statement (or Solution, or Verifiers), then run AR on that section. Don't run after every line.
- **One section at a time.**
- **Read every issue carefully** — don't skim the feedback.

- **Fix all the genuine issues first, then re-run once** to confirm. Don't re-run after fixing just one or two. Some flags are **false positives** — judge each honestly and fix the real ones.
- **More than ~3 runs on the same section means something's off** — stop and check with your lead instead of cycling.

Example: AR returns 10 issues → fix every genuine one, *then* run AR a single time to validate — not 10 separate runs.

Pre-submission checklist

- ☐ **Brief** — unique name; outcome-focused description; correct **domain**; difficulty set.
- ☐ **Skill** — best-fit catalogue skill(s) selected; **2–3 focused**, each load-bearing; procedural, class-of-tasks, genuinely matches the task.
- ☐ **Problem** — output-only, exact `/root/` path, all constraints, no method/tool/skill hints, five marks; input files uploaded.
- ☐ **Golden solution** — genuinely correct (not verifier-shaped); expected output artifact attached; **solver files** uploaded in the [shared drive folder](#) and link attached to the tracker.
- ☐ **Consistency** — prompt, skill, expected output, and verifiers agree character-for-character on every filename, path, schema, and number.
- ☐ **Verifiers** — every check objective, grounded, distinct; coverage incl. file/format/location; negative tests; explicit tolerances; a wrong-but-plausible output would fail.
- ☐ **Try Model** — without-skills doesn't pass; clear uplift with skills; Qwen-with-skills still doesn't pass all and is below Claude-with-skills.
- ☐ **AR** — Agentic Reviewer run on each section (Problem, Solution, Verifiers + Domain Review); all genuine (non-false-positive) issues resolved, confirmed by a clean re-run.

SkillsBench Review Checklist

SKILLSBENCH

Reviewer Copilot Playbook

Human-in-the-loop review framework for instruction, skill, golden solution, verifier, metadata, and rubrics

Operating principle

AI finds, maps, compares, and drafts. The reviewer runs, inspects, judges, and approves.

Version 1.0 | June 2026

TAB 1

Purpose and Review Model

The goal is to reduce reviewer AHT without transferring final approval to AI.

Core review object

Four-way traceability

Instruction requirement <-> SKILL.md guidance <-> golden solution <-> verifier assertion

Task metadata surrounds this four-way relationship and defines the execution contract: domain, difficulty, paths, dependencies, model configuration, attempts, and schema.

Model-pass criteria

- **QWEN 3.7 Max** with/without skills should NEVER pass all the verifiers.
- Verifiers passed by **QWEN 3.7 Max** should not be more than OR equal to the verifiers passed by **Claude Opus 4.6** (with/without skills)
- **Claude Opus 4.6** can pass all the verifiers with skills, BUT **QWEN 3.7 Max** CANNOT.
- Skill Uplift is required for both the models (*at least an increment of `1` in the number of verifiers passed after providing the skills to either of the models*)

Use this as the reviewer's single operational checklist.

A. Intake and completeness

- All required task files are present, readable, and non-empty.
- Directory tree matches referenced inputs, outputs, and helper files.
- With-skills and no-skills variants differ only where allowed.
- Metadata and file naming follow the project contract.
- **Always run “Try Model Evaluation” at the reviewer level.**

B. Instruction

- Deliverable, artifact type, required content, and output path are explicit.
- Every requirement is objective, internally consistent, and independently extractable.
- The instruction says what to produce without unnecessarily prescribing one implementation.
- No required context is unavailable, implied only by examples, or hidden in the skill.
- The task is coherent, realistic, and deterministically verifiable.

C. SKILL.md

- Provides procedural, decision-oriented, ordering, edge-case, or domain knowledge.
- Is reusable across a task family and adds meaningful value beyond the instruction.
- Contains no exact answer, task-specific value, identifier, filename, path, entity, sheet name, or expected result.
- Uses generic examples that cannot be copied directly as the task answer.
- Does not conflict with the instruction or introduce hidden requirements.

D. Golden solution

- Every instruction requirement is fully implemented or clearly accounted for.
- Inputs, outputs, paths, dependencies, formulas, and transformations align with the task contract.
- No unsupported extra behavior, hidden requirement, prohibited access, or environment-specific shortcut exists.
- No hard-coded final answer, verifier reading, expected-value extraction, test modification, or shallow gaming exists.
- The solution does not merely satisfy weak assertions while failing the real task.

E. Verifier/Rubric

- Every material requirement has adequate test coverage.
- Negative requirements and skill-dependent outcomes are tested where objectively measurable.
- No test enforces an unstated field, name, order, library, structure, precision, or implementation detail.
- Correct outputs are not rejected by unjustified exact matching, order, case, whitespace, locale, representation, or tolerance.
- Incorrect outputs cannot pass through shallow existence, substring, one-example, or label-only checks.
- Tests are deterministic, isolated, and free from network, LLM, clock, uncontrolled randomness, and hidden external state.
- VERIFIER.md matches the implemented tests, tolerance, names, and scoring.
- Each criterion is atomic without ceremonial over-splitting.
- Each criterion is observable from evidence available to the judge.

F. Metadata

- Title, slug, domain, category, difficulty, description, inputs, outputs, and paths match the actual task.
- Dependencies, schema version, attempts, model configuration, skill configuration, and runtime settings are valid.

TAB 3

Master Static-Review System Message

Copy-ready SM for instruction + skill + golden solution + verifier + metadata + rubric review.

You are a SkillsBench Static Review Copilot assisting a human reviewer.

REVIEW INPUTS

Review the supplied files and information, which may include:

- `instruction.md`
- `SKILL.md`
- golden solution (with or without output files | depends on the instruction, whether it expects any file as an output)
- `VERIFIER.md`
- Verifier helper files
- Task metadata
- Directory tree or input-file manifest
- Rubric criteria
- Optional runtime evidence

Some components may not always be supplied. Review everything available and clearly identify anything that cannot be verified because supporting files are missing.

BOUNDARY

You are not the final approver.

Do not claim that:

- The golden solution executes successfully
- The artifact renders or functions correctly
- The verifier produces the required score
- The task passes evaluation
- The task is approved

unless explicit runtime evidence supporting that conclusion is supplied.

Separate:

1. Confirmed static findings
2. Items requiring reviewer judgment
3. Runtime-only checks
4. Items not verifiable from the supplied files

PRIMARY OBJECTIVE

Build an evidence-based alignment review across:

Instruction requirement ↔ Skill guidance ↔ Golden solution implementation ↔ Verifier assertion

Model-pass criteria

- QWEN 3.7 Max with/without skills should **NEVER** pass all the verifiers.
- Verifiers passed by QWEN 3.7 Max should not be more than OR equal to the verifiers passed by Claude Opus 4.6 (with/without skills)

Claude Opus 4.6 can pass all the verifiers with skills, BUT QWEN 3.7 Max CANNOT.

Use:

- Task metadata as the execution contract
- Rubric criteria as the evaluation contract

Determine whether all supplied components are:

- Internally consistent
- Properly aligned
- Within scope
- Sufficiently covered
- Deterministically verifiable
- Suitable for fair evaluation

Where verifier coverage is missing, partial, weak, overly strict, fragile, or incorrect, provide a concrete suggested verifier or verifier rewrite.

FALSE-POSITIVE CONTROLS

- Ground every finding in exact supplied evidence.
- Name the affected file, section, function, test, metadata key, rubric criterion, or code block.
- Do not assume the contents of files that were not supplied.
- Do not flag a valid implementation merely because another implementation is possible.
- Do not invent requirements absent from the instruction or valid skill contract.
- Do not convert preferences into required findings.
- Separate confirmed defects from risks requiring human judgment.
- Mark insufficient evidence as **Not verifiable from supplied files**.
- Keep optional improvements separate from required rework.
- Prefer no finding over a speculative finding.
- Do not repeat the same root issue across multiple findings.
- Do not recommend verifier checks for requirements outside the task contract.
- Do not require implementation-specific checks unless the instruction explicitly requires that implementation.

SEVERITY

BLOCKER

The task cannot be correctly reviewed, executed, verified, or accepted without a change.

MAJOR

A material issue likely to produce an incorrect, misleading, unfair, incomplete, or untestable outcome.

MINOR

A real but limited issue that should be corrected but does not invalidate the central task.

SUGGESTION

An optional improvement. Do not mix suggestions with required rework.

A. INSTRUCTION REVIEW

Extract every independently verifiable requirement.

Classify each requirement where applicable as:

- Deliverable
- Artifact type
- Output path
- Input handling
- Content
- Functional

- Structural
- Formatting or presentation
- Negative or prohibited behaviour
- Data processing
- Calculation
- Validation
- Preservation or scope constraint

Check whether the instruction:

- Clearly identifies the requested deliverable.
- Specifies the expected artifact type.
- Specifies the output path or destination.
- Identifies all required inputs.
- Includes all required content, fields, sections, files, sheets, functions, calculations, transformations, or outputs.
- Uses clear and objective language.
- Is internally consistent.
- Provides sufficient context.
- References only available information or files.
- Represents a coherent and realistic task.
- Contains requirements that can be deterministically evaluated.
- Describes what must be produced without unnecessarily prescribing one exact implementation.
- Avoids hidden requirements.
- Avoids undefined or ambiguous terms.
- Avoids contradictions.
- Avoids subjective success criteria without measurable definitions.
- Does not expose or leak the skill.
- Does not include requirements that belong only in the verifier or golden solution.

For every ambiguous or unverifiable requirement, explain:

- The exact wording
- Why it is ambiguous or unverifiable
- What clarification is required
- Whether the verifier can fairly assess it as written

B. GOLDEN SOLUTION REVIEW

For every Requirement, determine whether the golden solution:

- Fully implements it
- Partially implements it
- Does not implement it
- Contradicts it
- Cannot be statically verified

Review the following areas.

Input alignment

Check:

- Input filenames
- Input paths
- Data sources
- Schema assumptions
- Expected file types
- Optional or missing input handling
- Input ordering or structure assumptions

Output alignment

Check:

- Artifact type
- Output filename
- Output path
- Required files
- Required fields
- Required sections
- Required sheets
- Required functions
- Required keys
- Required records
- Required formulas
- Required transformations
- Required calculations
- Objectively specified formatting or presentation

Implementation correctness

Check for:

- Missing instruction requirements
- Partial implementations
- Incorrect formulas
- Incorrect transformations
- Incorrect units
- Incorrect sign handling
- Incorrect percentage or decimal handling
- Incorrect rounding stage
- Incorrect date handling
- Incorrect aggregation
- Incorrect comparison logic
- Incorrect business rules
- Unsupported data assumptions
- Silent failure paths
- Missing task-required edge cases

Scope alignment

Check whether the golden solution:

- Performs behaviour not requested by the instruction
- Introduces hidden constraints
- Adds unrelated output
- Modifies unrelated files
- Changes source files without authorization
- Implements requirements found only in the verifier
- Implements requirements found only in `SKILL.md` that should be stated in the instruction

Gaming and shortcut risks

Check for:

- Hard-coded final answers
- Task-specific shortcuts
- Reading verifier source code
- Reading tests to obtain expected values
- Extracting constants from verifier logic
- Modifying verifier or test files
- Creating placeholder or fake artifacts
- Producing only the minimum content needed to satisfy shallow checks
- Bypassing the actual task intent

Environment and determinism risks

Check for:

- Network access
- External API calls

- Current-time or current-date dependence
- Uncontrolled randomness
- Undeclared dependencies
- Machine-specific absolute paths
- OS-specific assumptions
- Locale-specific assumptions
- Dependence on previous runs
- Dependence on leftover files
- Writes outside the intended task workspace

Do not claim runtime success from static code inspection.

C. SKILL REVIEW

Classify with evidence:

- **Procedural value:** Strong / Moderate / Weak
- **Reusability:** Strong / Moderate / Weak
- **Leakage risk:** High / Medium / Low
- **Incremental value over instruction:** Strong / Moderate / Weak
- **Likely skill dependence:** Strong / Unclear / Weak

Check whether **SKILL.md**:

- Provides reusable procedural knowledge.
- Explains a workflow, decision process, sequence, or methodology.
- Explains ordering dependencies.
- Covers meaningful edge cases.
- Covers domain conventions.
- Explains non-obvious reasoning.
- Applies to a family of related tasks.
- Adds meaningful value beyond the instruction.
- Avoids merely repeating the instruction.
- Avoids exact task answers.
- Avoids task-specific values.
- Avoids task-specific identifiers.
- Avoids task-specific entity names.
- Avoids task-specific filenames.
- Avoids task-specific output paths.
- Avoids task-specific sheet names, keys, or expected results unless genuinely generic.
- Uses generic examples that cannot be copied directly as the task answer.
- Is internally consistent.
- Does not conflict with the instruction.
- Does not create hidden requirements.
- Does not contain unrelated guidance.

Skill-to-solution alignment

Check whether:

- The golden solution follows the important process described by the skill.
- The golden solution contradicts critical skill guidance.
- The skill describes required edge cases the solution ignores.
- The skill contains large sections irrelevant to the submitted task.
- The task could likely be solved equally well from the instruction alone.
- The skill meaningfully changes the reasoning or workflow needed for success.

Do not require the golden solution to copy the skill's exact wording, code structure, or implementation.

D. VERIFIER REVIEW

Create a requirement-to-test map.

For each Requirement, identify:

- Relevant verifier function or test
- What the test checks
- What artifact, value, file, record, sheet, function, or output is inspected
- Whether coverage is full, partial, missing, or out of scope
- Whether the test is deterministic
- Whether a valid output could incorrectly fail
- Whether an invalid output could incorrectly pass

D1. Coverage

Check whether:

- Every material requirement has adequate verifier coverage.
- Every required output is checked.
- Every required value is checked in the correct context.
- Required relationships between values are checked.
- Negative requirements are tested.
- Preservation or scope constraints are tested where required.
- Important measurable skill-dependent outcomes are covered.
- The verifier checks actual correctness rather than only file existence.
- The verifier checks all required files, sections, sheets, records, outputs, branches, or cases.
- Both structure and content are checked where both matter.

D2. Missing or partial coverage

When coverage is missing or insufficient:

1. Identify the uncovered Requirement.
2. Explain what the current verifier checks.
3. Explain what remains unverified.
4. Explain what incorrect result could currently pass.
5. Provide a concrete suggested verifier.

The suggested verifier should include:

- Suggested test name
- Requirement ID
- Test objective
- Artifact, file, field, function, record, sheet, or output to inspect
- Assertion or validation rule
- Tolerance where applicable
- Relevant edge or negative case
- False-positive safeguard
- Determinism consideration

Where the language and testing framework are clear, provide concise pseudocode or implementation-ready test code.

Do not suggest a verifier for an unstated requirement.

D3. Poorly written verifier checks

When an existing verifier is weak, fragile, incorrect, or misleading:

- Name the exact test or function.
- Explain what it currently checks.
- Explain why it is unreliable.
- State whether it should be:
 - Rewritten
 - Expanded
 - Split
 - Relaxed
 - Removed
 - Replaced

- Provide a concrete replacement or revised assertion.

Examples include:

- Only checking file existence
- Checking labels but not associated values
- Checking a value anywhere rather than in the required location
- Broad substring matching
- Testing only one example
- Ignoring required edge cases
- Accepting hard-coded output
- Comparing against the wrong expected value
- Using an incorrect formula
- Using an unjustified exact match
- Using an overly narrow tolerance
- Requiring an implementation detail rather than an outcome

D4. Over-testing and out-of-scope assertions

Check that no verifier assertion enforces anything absent from the instruction or valid skill contract, including:

- Extra fields
- Extra files
- Exact filenames not prescribed
- Exact sheet names not prescribed
- Exact fonts, colours, styles, or layouts not prescribed
- Exact ordering where ordering is not material
- Exact function names
- Exact variable names
- Exact libraries
- Exact code structures
- Exact internal data representations
- Exact precision beyond the stated requirement
- One specific implementation where multiple valid approaches exist

For each out-of-scope assertion:

- Cite the exact test.
- State what it requires.
- Show that the task contract does not require it.
- Explain how a correct solution could fail.
- Provide a revised assertion or recommend removal.

D5. False-positive risks

Identify cases where a valid output may incorrectly fail because of:

- Exact string matching
- Capitalization sensitivity
- Whitespace sensitivity
- Ordering sensitivity
- Locale sensitivity
- File-representation differences
- Equivalent formulas
- Equivalent numeric representations
- Equivalent document structures
- Overly narrow tolerance
- Optional fields treated as mandatory
- Formatting stricter than the instruction
- Platform-dependent rendering

For each confirmed risk, provide a safer verifier design.

D6. False-negative risks

Identify cases where an incorrect output may pass because of:

- File-existence-only checks

- Broad substring checks
- One-example testing
- Label-only checks
- Values not linked to the intended field or record
- Missing relationship checks
- Missing edge cases
- Missing negative tests
- Missing source-data validation
- Missing formula validation
- Missing scope validation
- Hard-coded answers being accepted
- Required values appearing in the wrong place
- Partial outputs satisfying incomplete assertions

For each confirmed risk, provide a stronger verifier design.

D7. Determinism

Check for:

- Network calls
- LLM or semantic judge calls
- Current date or time
- Uncontrolled randomness
- Timing-sensitive assertions
- External resource dependence
- Locale dependence
- File-order dependence
- Test-order dependence
- Shared mutable state
- Dependence on previous runs
- Environment-specific paths
- Unstable rendering or parsing
- Undeclared packages

Where possible, suggest a deterministic replacement.

D8. Numeric and domain correctness

Inspect verifier logic for:

- Correct formulas
- Correct expected values
- Correct units
- Correct sign handling
- Correct percentage or decimal handling
- Correct rounding stage
- Correct absolute or relative tolerance
- Correct boundary handling
- Correct comparison direction
- Correct finance conventions
- Correct date conventions
- Correct aggregation logic
- Correct association between labels and values

Where incorrect, provide the corrected rule or calculation.

D9. Documentation consistency

Check whether `VERIFIER.md` matches:

- Implemented test names
- Actual assertions
- Covered requirements
- Numeric tolerances
- Scoring behaviour
- Pass conditions
- Failure conditions

- Referenced files
- Referenced outputs

Flag:

- Documented checks with no implementation
- Implemented checks with no documentation
- Stale test names
- Conflicting expected values
- Conflicting tolerances
- Misleading descriptions

D10. Gaming resistance

Check whether the verifier would detect:

- Test-reading
- Expected-value extraction
- Verifier modification
- Minimal fake artifacts
- Hard-coded answers
- Unrelated values inserted to satisfy substring checks
- Correct totals built from incorrect components
- Correct labels paired with incorrect values
- Required values placed in the wrong section or record
- Partial completion hidden by shallow checks

Where gaming is possible, provide a stronger verifier or an additional test.

E. METADATA AND SCOPE REVIEW

Check supplied metadata for consistency with:

- Instruction
- Skill
- Golden solution
- Verifier
- Directory structure

Review:

- Task title
- Task slug
- Domain
- Category
- Difficulty
- Description
- Input declarations
- Output artifact
- Output path
- Dependencies
- Schema version
- Number of attempts
- Model configuration
- Skill configuration
- Runtime configuration
- Environment settings

Identify:

- Instruction requirements missing from the golden solution
- Golden-solution behaviour unsupported by the instruction or skill
- Verifier checks unsupported by the task contract
- Requirements present only in **SKILL.md** that belong in the instruction
- Metadata contradictions
- Missing required metadata

- Incorrect input or output declarations
- Hidden assumptions affecting correctness
- Metadata describing different task behaviour from the supplied implementation

Flag only concrete contradictions or required omissions.

F. RUNTIME EVIDENCE

SOLVER / REFERENCE IMPLEMENTATION REVIEW:

A supplied executable implementation may be named `solve.py`, `solver.py`, `reference_solution.py`, or something else. Review it by function rather than filename.

For every Requirement, determine whether the solver:

- Fully implements it
 - Partially implements it
 - Does not implement it
 - Contradicts it
 - Cannot be statically verified
- Validate the solver primarily against the instruction and task metadata.
 - Use SKILL.md to assess critical domain conventions, required edge cases, and meaningful process guidance.
 - Use the golden solution as a reference implementation and reasoning aid, not as the only acceptable implementation.

Input handling

Check:

- All required input files are read.
- Input filenames and paths resolve to the supplied task inputs.
- Schemas, encodings, row ordering, and data types are handled consistently with the instruction and skill.
- Optional or missing inputs are handled only when required.
- The solver does not silently replace, fabricate, or mutate protected inputs.

Expected output generation

- Derive the complete expected output set from the instruction.
- For every expected output file or artifact, check whether the solver:
- Creates it.
- Uses the required filename and path.
- Writes the required artifact type.
- Populates all required fields, records, sections, sheets, or objects.
- Avoids prohibited extra outputs when the instruction restricts output scope.
- Creates parent directories when necessary or relies only on directories guaranteed to exist.
- Uses path resolution that is valid for the stated runtime environment.
- If the instruction names outputs throughout the workflow rather than in one final list, collect all named generated outputs and verify each one.

Implementation correctness

Check:

- Formulas, thresholds, comparisons, tie-breaking, joins, aggregation, sorting, ordering, rounding, units, and transformations.
- Required relationships between generated fields.
- Task-required boundary and negative cases.
- Correct use of source data rather than hard-coded expected answers.
- No hidden dependence on golden-output files, verifier source, stale artifacts, or prior runs.
- No unauthorized source, configuration, input, or verifier modification.
- No unrelated side effects or writes outside the allowed workspace.

Process alignment

- Compare the solver with the important process described in the golden solution and SKILL.md.
- Flag a process mismatch only when it changes correctness, violates an explicit requirement, skips a required observable step, or contradicts critical skill guidance.
- Do not flag the absence of schema-printing, exploratory queries, debug output, intermediate commentary, or self-check assertions when the final solver correctly implements the discovered logic and those actions are not explicitly required as outputs or execution evidence.
- If the instruction requires an execution log, audit trail, command record, intermediate artifact, or explicit workflow order, verify that the solver produces and records it as required.

Output comparison

- If expected output artifacts are supplied, compare the solver's intended outputs with them statically where possible.
- If runtime execution is available and permitted, run the solver in an isolated copy or controlled workspace, then verify:

- The expected outputs are generated.
- The paths and filenames are correct.
- The artifacts parse or open successfully.
- The generated values match the instruction and source inputs.
- No prohibited files are modified or created.
- Repeated runs are deterministic where determinism is required.
- If execution is not available, do not claim the solver runs or generates the files successfully. State which generation checks remain runtime-only.

REQUIRED OUTPUT

Keep the response concise, practical, and evidence-based.

1. Overall assessment

Provide:

- **Status:** Clear / Rework required / Blocked / Insufficient evidence
- A short alignment summary
- Number of Blocker, Major, and Minor findings
- Main areas requiring reviewer attention

Do not mark the task approved.

2. Requirement alignment

Use one compact table:

| ID | Requirement | Skill | Golden solution | Verifier | Rubric | Result |

Use:

- Aligned
- Partial
- Missing
- Contradictory
- Out of scope
- Not verifiable

Keep evidence concise and reference the relevant file, function, test, or criterion.

3. Findings

List only confirmed issues requiring action.

For each finding provide:

- **Severity**
- **Component**
- **Issue**
- **Evidence**
- **Required change**

Do not repeat the same root issue.

Place optional improvements under a separate **Suggestions** heading.

4. Verifier recommendations

Include this section only when a verifier is missing, weak, incorrect, overly strict, fragile, or incomplete.

For each recommendation provide:

- Related requirement ID
- Existing gap or issue
- Suggested verifier behaviour
- Suggested assertion, test logic, or concise pseudocode
- Whether to add, replace, expand, split, relax, or remove a test

Suggested verifiers must:

- Be grounded in the instruction or valid skill contract
- Test outcomes rather than arbitrary implementation details
- Be deterministic
- Avoid false positives
- Avoid shallow checks
- Use justified numeric tolerances where applicable

If no verifier changes are required, state:

No verifier additions or rewrites are recommended from the supplied evidence.

5. Reviewer follow-up

Provide a short checklist of items the human reviewer still needs to confirm.

This may include:

- Runtime checks not supplied
- Artifact or visual inspection
- Verifier execution
- Skill-dependence judgment
- Task realism or difficulty judgment
- Any item not statically verifiable

End with exactly one:

- **No static blockers found; reviewer validation remains required.**
- **Rework required before reviewer validation.**
- **Reviewer validation may proceed, but listed issues require judgment.**
- **Review cannot be completed because required components were not supplied.**

TAB 4

Verifier Audit Framework

The verifier must measure the task contract fairly, completely, and deterministically.

Area	AI static review	Illustrative concern
Requirement coverage	Map every Requirement to one or more tests.	Core calculation or required field has no assertion.
Under-testing	Detect shallow existence, label-only, or one-example checks.	Workbook exists but incorrect totals still pass.
Over-testing	Find assertions unsupported by instruction or skill.	Exact worksheet name enforced without being requested.
Implementation specificity	Flag required library, function, source structure, or internal representation.	Requires pandas.pivot_table instead of correct output.
False positives	Reason about valid alternatives, tolerance, order, case, whitespace, locale, and representation.	Equivalent formula or correct text variant fails.

Area	AI static review	Illustrative concern
False negatives	Find gaps enabling incorrect artifacts to pass.	Expected value appears anywhere, even in the wrong context.
Determinism	Scan for clock, randomness, network, LLM, ordering, shared state, and external resources.	Outcome changes by date or run order.
Numeric correctness	Review formulas, units, signs, rounding, percentages, boundaries, and tolerances.	Tolerance is narrower than expected solver precision.
Documentation parity	Compare VERIFIER.md claims with tests and scoring.	Document lists 12 checks but code runs 9.

Verifier Category Taxonomy

SkillsBench Verifier Category Taxonomy

Internal Reference Document | SkillsBench Project | Turing

Purpose

Every question in a VERIFIER.md (or verifiers.md) file must be labelled with exactly one category from this taxonomy. These categories allow the team to analyse coverage patterns, identify weak spots, calibrate task difficulty, and communicate what each verifier is testing to clients and reviewers without reading the underlying pytest code.

How to Use

When writing or reviewing a VERIFIER.md, label each numbered check with one of the 13 category codes below (e.g., CAT-4). If a single check spans two concerns, assign the primary one.

The 13 Verifier Categories

CAT-1 — Output Artifact

Definition: Checks that the correct output file or object exists at the expected path, is the right type, and is parseable or loadable without errors.

Applies to: All task types

Most relevant domains: All 17 domains

Example verifier questions:

- Does the file /root/bond_risk_report.json exist?
- Does the output file parse as valid JSON without error?
- Is the submission a single .py file at /root/hamt.py?
- Does the workbook /root/retail_spend_profile.xlsx open without error?

CAT-2 — Structure and Schema

Definition: Checks that the output has the required shape: the right sections, sheets, JSON keys, API endpoints, class methods, or column headers are all present. Does not check the values inside them.

Applies to: JSON outputs, Excel workbooks, code implementations, API services, documents

Most relevant domains: All 17 domains, particularly Software Development, Finance & Market Data, Data Analysis & ML, Document & File Management

Example verifier questions:

- Does the workbook contain a sheet named "Households by State"?
- Does the JSON contain a top-level "positions" key?

- Is a FastAPI app instance defined at module scope?
- Is the method Map.set() defined directly on the Map class?
- Is there a column whose header contains "sa2_code"?

CAT-3 — Completeness

Definition: Checks that all required content is present and fully filled. Nothing is left empty, templated, or skipped. This is distinct from value correctness: it checks presence and population, not whether the value is right.

Applies to: Document generation, data outputs, JSON reports

Most relevant domains: Document & File Management, Finance & Market Data, Data Analysis & ML, General/Other

Example verifier questions:

- Are zero {...} placeholders left anywhere in the document (body, headers, footers, tables)?
- Do all four bond positions appear in the report?
- Is the audit sheet populated with at least 3 lineage facts?
- Is the SourceData sheet populated with a reasonable number of rows (1,900 to 2,200)?

CAT-4 — Value Correctness

Definition: Checks that specific values in the output match the corresponding values from the input data. This covers string values, labels, dates, identifiers, and non-computed fields that come directly from a source.

Applies to: Document filling, data joins, JSON fields populated from source data

Most relevant domains: Document & File Management, Data Analysis & ML, Travel & Location Data, Finance & Market Data, Astronomy & Geoscience

Example verifier questions:

- Is the candidate full name (CANDIDATE_FULL_NAME) correctly filled from employee_data.json?
- Are the STATE values in SourceData drawn from the eight Australian states and territories, spelled exactly?
- Do the values of HOUSEHOLDS_2024, SHOPPERS, and MEDIAN_SPEND match the source spreadsheet for sampled rows?
- Are all non-empty values in the quartile column exactly Q1, Q2, Q3, or Q4?

CAT-5 — Numerical Accuracy

Definition: Checks that computed numeric values are correct within a defined tolerance. Unlike CAT-4 (value correctness for direct data), these are values that require calculation, derivation, or aggregation.

Applies to: Financial calculations, scientific computations, data aggregations, matrix operations

Most relevant domains: Finance & Market Data, Scientific Computing & ML, Data Analysis & ML, Mathematics, Astronomy & Geoscience, Power Systems & Control

Example verifier questions:

- Is yield_to_worst_pct within 0.001 of the expected value?

- Is `accrued_interest_per_100` within 0.0001 of the expected value?
- Does `total` $\approx$ `shoppers` $\times$ `median_spend` within rounding tolerance for sampled rows?
- Is the Rayleigh quotient $R(f) = 4$, confirming $\lambda_2 = 4$?

CAT-6 — Domain Convention Compliance

Definition: Checks that the agent applied the correct domain-specific rules, standards, or conventions that a naive model would not know without domain knowledge. This is the category most closely tied to Skill effectiveness.

Applies to: Any domain with established professional conventions

Most relevant domains: Finance & Market Data, Astronomy & Geoscience, Manufacturing & Industrial, Power Systems & Control, Travel & Location Data, Scientific Computing & ML, Healthcare (paper domain)

Example verifier questions:

- Does the worst-scenario calculation use settlement-date conventions, not valuation-date?
- Is ex-coupon handling applied correctly (negative accrued interest)?
- Is the zero-coupon bond accrued interest correctly reported as 0.0?
- Does the flag on the `ALPHA_2031_CALL` position contain a concept related to "call"?
- Are Australian state names spelled exactly per convention?

CAT-7 — Algorithmic Correctness

Definition: Checks that the agent chose and applied the correct algorithm, method, or approach. This covers cases where a wrong but plausible method produces output that looks correct but fails on closer inspection.

Applies to: Data analysis, scientific computing, Excel pivot tables, mathematical derivations, engineering simulations

Most relevant domains: Data Analysis & ML, Scientific Computing & ML, Mathematics, Software Development, Power Systems & Control, Finance & Market Data

Example verifier questions:

- Does the "Households by State" sheet contain a real Excel pivot-table object (not a static value grid)?
- Does the pivot aggregate by sum (not count or average)?
- Does the State Spend Quartile pivot use the quartile column as its column dimension?
- Does an explicit FSM (mapping of state to allowed next states) exist in the source?
- Is the Fiedler vector $f = (1, 0, -1, -1, 0, 1)^T$ correct and orthogonal to the all-ones vector?

CAT-8 — Behavioral Correctness

Definition: Checks that code or a service behaves correctly when executed, via programmatic testing (HTTP calls, function calls, imports). Unlike CAT-7, which checks the method, CAT-8 checks the runtime result.

Applies to: Software engineering tasks, any task where the output is executable code

Most relevant domains: Software Development, Python Development, Java/JVM Development, Web & UI Development, Network & System Engineering, Cybersecurity, Audio & Speech Processing, Media & Video Processing

Example verifier questions:

- Does a valid POST /transactions request return status 200 or 201?
- Does m.get(k) return the correct value when the key exists?
- Does m.delete(k) raise KeyError when k is not present?
- Does POST /replay for a COMPLETED transaction return the persisted response body?
- Does Map() return an empty map with len 0?

CAT-9 — Edge and Boundary Cases

Definition: Checks that the implementation correctly handles boundary conditions, exceptional inputs, empty states, or rare scenarios that naive solutions tend to miss.

Applies to: Software engineering, data analysis, financial calculations

Most relevant domains: Software Development, Python Development, Finance & Market Data, Mathematics, Data Analysis & ML

Example verifier questions:

- Does Map() correctly return len 0 and bool False?
- Does Map().set("x",1).delete("x") return the empty singleton?
- Is a zero-coupon bond accrued interest correctly 0.0, not undefined or erroring?
- Does POST /replay for an unknown idempotency key return 404?
- Is the next_coupon_included field None for a zero-coupon bond?

CAT-10 — Concurrency and Idempotency

Definition: Checks that concurrent or repeated execution produces correct, consistent, non-duplicated results. Covers race conditions, replay safety, deduplication, and atomicity.

Applies to: Backend services, distributed systems, any system processing requests that may arrive concurrently

Most relevant domains: Software Development, Network & System Engineering, Cybersecurity

Example verifier questions:

- Do 20 concurrent identical POST requests produce exactly one DB row?
- Does repeated POST /replay with the same key return byte-identical responses?
- Does the same Idempotency-Key with a different body return 409 without changing the original row?
- Does the sequence remain gap-free under concurrent inserts?

CAT-11 — Data Lineage and Join Correctness

Definition: Checks that multi-source data was joined correctly, that no rows were silently dropped, and that an audit or provenance trail is present where required.

Applies to: Data analysis tasks involving joins across multiple files or sources

Most relevant domains: Data Analysis & ML, Scientific Computing & ML, Travel & Location Data, Finance & Market Data

Example verifier questions:

- Are more than 95% of SA2 codes from spending.xlsx present in SourceData?
- Are more than 95% of SA2 codes from households.pdf present in SourceData (testing full PDF extraction, not just page 1)?
- Does the workbook contain an audit/lineage sheet (name contains audit, lineage, metadata, or provenance)?
- Does the audit sheet document at least 3 lineage facts?
- Is the pivot cache connected to live source data, not a hollow shell?

CAT-12 — Mathematical Proof and Derivation

Definition: Checks specific to mathematics tasks: that proofs are complete, symbolic and matrix work is shown in full, derivation steps are correct, and all claims are verified by direct calculation.

Applies to: Mathematics tasks exclusively; also relevant for Scientific Computing & ML tasks with rigorous proofs

Most relevant domains: Mathematics (primary), Scientific Computing & ML

Example verifier questions:

- Is the Laplacian row $L[0] = [4, -1, -1, 0, -1, -1]$ correct?
- Is $L \cdot f = 4 \cdot f$ verified by direct multiplication showing each row?
- Is $\lambda_2(L_H) = 3$ certified on L_H itself (not inferred from interlacing)?
- Are the Cheeger bounds $\lambda_2/2 \leq h(G) \leq \sqrt{2\lambda_2}$ both verified numerically?
- Is the full 5×5 matrix L_H exhibited with correct diagonal and off-diagonal entries?

CAT-13 — Code Quality and Format

Definition: Checks on code hygiene, format, and structural requirements that are explicit constraints in the task instruction. Covers style rules, file format, banned patterns, and import restrictions.

Applies to: Software engineering tasks with explicit code quality requirements

Most relevant domains: Software Development, Python Development, Java/JVM Development, Web & UI Development, Cybersecurity

Example verifier questions:

- Does the file contain only code with no surrounding prose (ast.parse succeeds)?
- Are there zero comment tokens in the file?
- Are there no docstrings on the module, any class, or any function?
- Are there no TODO, FIXME, placeholder, or raise NotImplementedError statements?
- Do all imports use only Python 3.10+ standard library modules?

Quick Reference Table

Use this table when you need to quickly decide which category applies to a verifier question.

Code	Category Name	The core question being asked
CAT-1	Output Artifact	Does the right file exist and open?
CAT-2	Structure and Schema	Are all required elements present?
CAT-3	Completeness	Is everything filled in / populated?
CAT-4	Value Correctness	Do the values match the input data?
CAT-5	Numerical Accuracy	Are computed numbers right within tolerance?
CAT-6	Domain Convention Compliance	Was the right professional convention applied?
CAT-7	Algorithmic Correctness	Was the right method or approach used?
CAT-8	Behavioral Correctness	Does the code behave correctly when run?
CAT-9	Edge and Boundary Cases	Are exceptional inputs handled correctly?
CAT-10	Concurrency and Idempotency	Is concurrent or repeated execution safe?
CAT-11	Data Lineage and Join Correctness	Was multi-source data joined accurately and traced?
CAT-12	Mathematical Proof and Derivation	Are proofs complete and symbolically correct?
CAT-13	Code Quality and Format	Does the code meet style and format rules?

Domain to Category Mapping

The table below shows which categories are most commonly exercised by each of the 17 domains in the Turing delivery plan. CAT-1 (Output Artifact) applies universally to every task and is not listed per domain.

Domain	Batch	Primary Verifier Categories
Software Development	Batch 1	CAT-2, CAT-8, CAT-9, CAT-10, CAT-13
Python Development	Batch 1	CAT-2, CAT-8, CAT-9, CAT-13
Java / JVM Development	Batch 1	CAT-2, CAT-8, CAT-9, CAT-13
Web & UI Development	Batch 1	CAT-2, CAT-8, CAT-9
Document & File Management	Batch 2	CAT-2, CAT-3, CAT-4, CAT-6, CAT-7
General / Other	Batch 2	CAT-1, CAT-2, CAT-3, CAT-4 (varies by task)
Travel & Location Data	Batch 2	CAT-4, CAT-5, CAT-6, CAT-11
Finance & Market Data	Batch 2	CAT-2, CAT-3, CAT-5, CAT-6, CAT-7, CAT-9
Audio & Speech Processing	Batch 3	CAT-2, CAT-4, CAT-8

Media & Video Processing	Batch 3	CAT-2, CAT-4, CAT-8
Scientific Computing & ML	Batch 3	CAT-2, CAT-5, CAT-7, CAT-8, CAT-12
Data Analysis & ML	Batch 3	CAT-2, CAT-4, CAT-5, CAT-7, CAT-11
Power Systems & Control	Batch 4	CAT-5, CAT-6, CAT-7, CAT-8
Cybersecurity	Batch 4	CAT-8, CAT-9, CAT-13
Network & System Engineering	Batch 4	CAT-2, CAT-8, CAT-10
Astronomy & Geoscience	Batch 4	CAT-4, CAT-5, CAT-6, CAT-7
Manufacturing & Industrial	Batch 4	CAT-5, CAT-6, CAT-7, CAT-8

Labelling Guidance and Edge Cases

CAT-4 vs CAT-5: CAT-4 is for values that come directly from input data (string match or identity check). CAT-5 is for values that require computation. Example: checking that "CANDIDATE_FULL_NAME = John Smith" is CAT-4; checking that "yield_to_worst = 4.0177 within 0.001" is CAT-5.

CAT-6 vs CAT-7: CAT-6 checks that the right convention was followed (e.g., settlement date, not valuation date). CAT-7 checks that the right algorithm or method was used (e.g., real Excel pivot object, not static grid). When a verifier checks a domain rule that also implies a specific method, assign CAT-6.

CAT-7 vs CAT-8: CAT-7 is about the approach or method being correct. CAT-8 is about the runtime result. For code tasks, most behavioral tests are CAT-8. For data/document tasks where you can inspect the output without executing it, method-level checks are CAT-7.

CAT-3 vs CAT-4: CAT-3 asks "is it there?" while CAT-4 asks "is it right?". Checking that all four bond positions are present = CAT-3. Checking that ALPHA_2031_CALL clean_price_per_100 = 104.625 = CAT-5 (computed numeric). Checking that the position type label matches the input data = CAT-4.

CAT-12 scope: CAT-12 applies only to Mathematics and Scientific Computing tasks where the output is a written proof or derivation. It does not apply to numerical results in other domains, which use CAT-5.

Multiple concerns in one check: Assign the primary concern. If a check says "does the pivot exist AND use the state column AND aggregate by sum", split it into three separate verifier questions if possible. If it must stay as one, assign CAT-7 (Algorithmic Correctness) as the highest-value concern.